

Webcam Motion Detection Application: System Architecture Document

Document type	System Architecture & Design Specification
Version	1.0 (Draft for client review)
Date	October 4, 2026
Status	Proposed

1. Introduction

1.1 Purpose

This document describes the architecture and design of a web-based application that uses a webcam to detect motion. The application shows the live feed in **black and white while the scene is still**. When **movement is detected**, the feed switches to **full color** and an **alert** is raised.

1.2 Scope

In scope

- Live webcam access in the browser
- Real-time motion detection
- Visual state change (grayscale ↔ color)
- Alert message (on-screen, sound, optional browser notification)
- Event log and adjustable sensitivity

Out of scope (Phase 1): face/object recognition, multi-camera management, cloud video storage, mobile native apps. These are covered as future extensions in Section 18.

1.3 Definitions

Term	Meaning
Frame	A single still image captured from the video stream
Frame differencing	Comparing the current frame with the previous one to find changed pixels
Threshold	Minimum pixel change that counts as "different"
Cooldown	Short delay after motion stops before returning to black and white
Debounce	Ignoring very brief noise so that false alerts are avoided

2. Requirements

2.1 Functional Requirements

ID	Requirement
FR-1	The system shall request camera permission and display the live webcam feed.
FR-2	When no motion is present, the feed shall be displayed in black and white.
FR-3	When motion is detected, the feed shall switch to color immediately.
FR-4	When motion is detected, the system shall show an alert message.
FR-5	After motion stops for a configurable period, the feed shall return to black and white.
FR-6	The user shall be able to adjust sensitivity and cooldown.
FR-7	The system shall keep a log of motion events (time, duration, intensity).
FR-8	The user shall be able to start, pause and stop monitoring.
FR-9	(Optional) The system shall capture a snapshot when motion starts.
FR-10	(Optional) The system shall send the event to a backend for storage and notifications.

2.2 Non-Functional Requirements

Category	Requirement
Performance	Detection latency under 200 ms; at least 15 FPS on a mid-range laptop
Reliability	Recover automatically if the camera stream drops
Privacy	Video is processed locally in the browser by default; no frames leave the device unless the user enables it
Usability	One-click start, clear status indicator, readable alert
Compatibility	Latest Chrome, Edge, Firefox, Safari (desktop and mobile browsers)
Security	HTTPS only (required by browsers for camera access)
Maintainability	Modular components with clear interfaces and unit-testable logic
Scalability	Phase 1 is client-only; the optional backend scales horizontally

2.3 Constraints and Assumptions

- Browsers only expose the camera on **HTTPS** (or localhost).
- The user must grant camera permission.

- Lighting is reasonably stable; sudden lighting changes may trigger false positives (mitigated in Section 8).

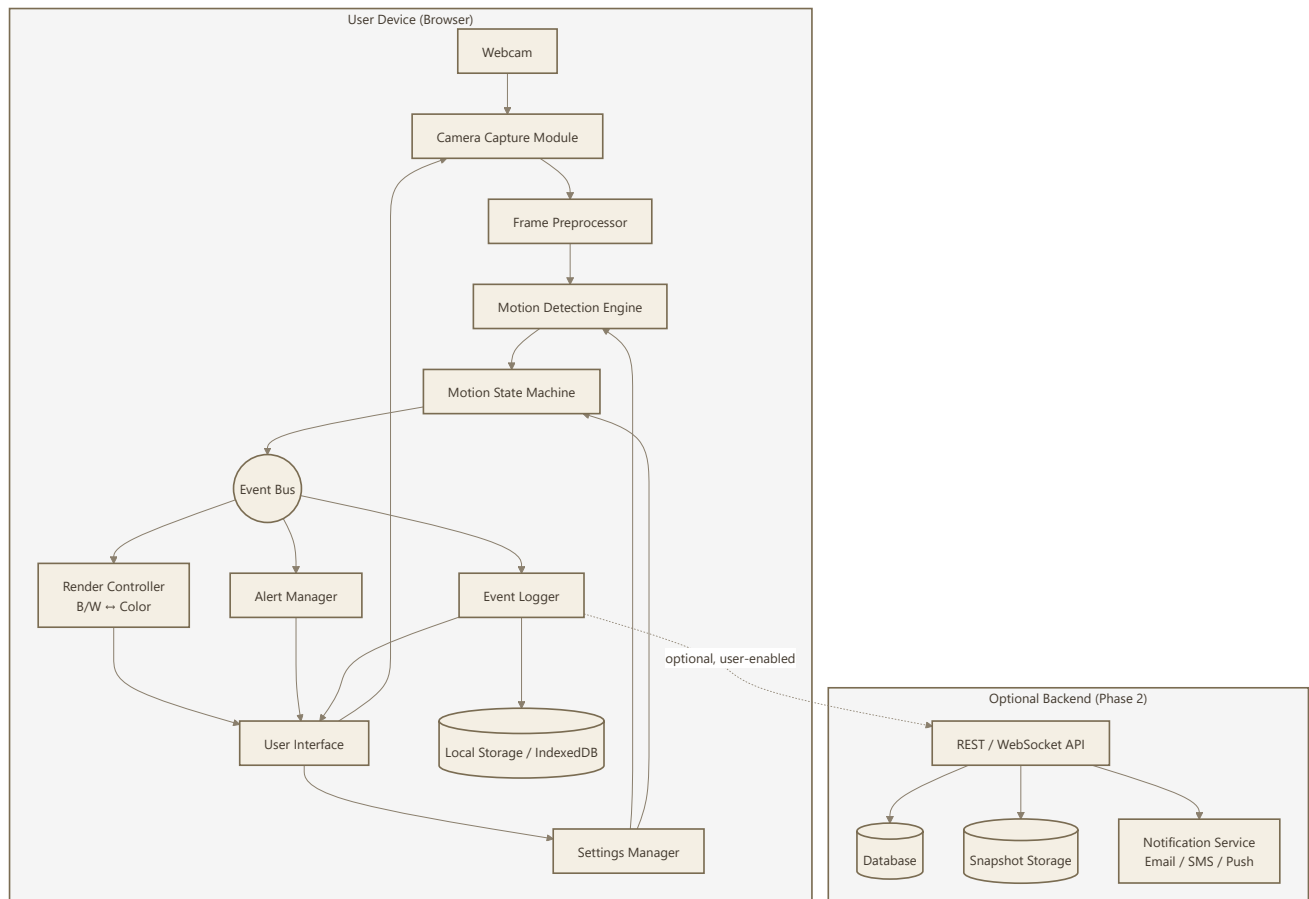
3. Architecture Overview

3.1 Architectural Style

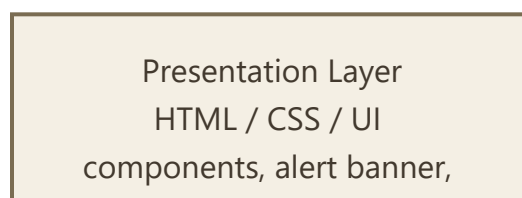
A **client-side, event-driven, pipeline architecture**. Video frames flow through a processing pipeline (capture → preprocess → detect → decide → render/alert). Components communicate through an internal **event bus**, so each module stays independent and replaceable.

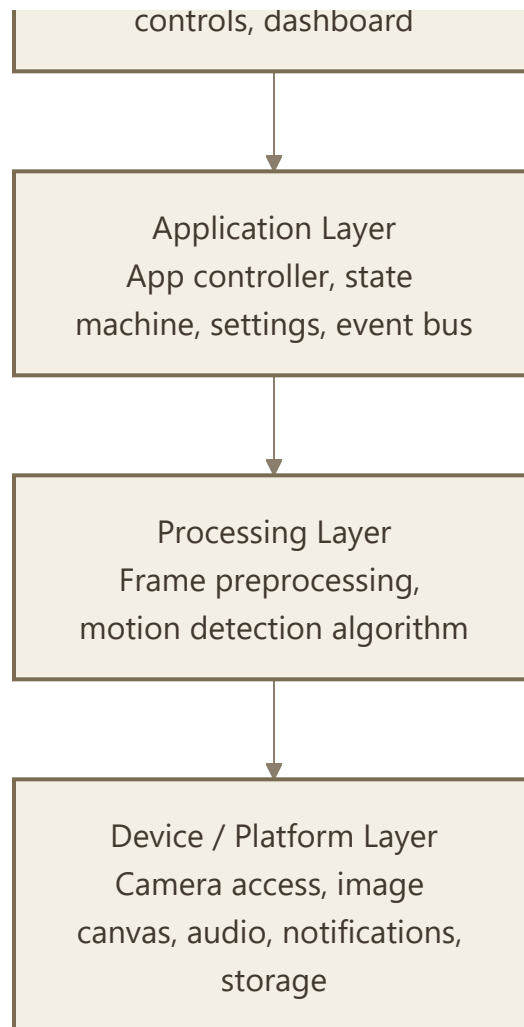
Why client-side? Motion detection by frame differencing is lightweight. Running it in the browser gives zero network latency, strong privacy, no server video costs, and works offline after the first load.

3.2 High-Level Architecture



4. Layered View





Layer	Responsibility
Presentation	Displays video, status, alert banner, controls and event history
Application	Coordinates modules, owns the motion state, manages configuration
Processing	Pure image-processing logic (no UI, easily testable)
Platform	Browser APIs used to reach hardware and storage

5. Component Design

5.1 Component Summary

#	Component	Responsibility	Key Inputs	Key Outputs
1	Camera Capture Module	Request permission, open stream, handle errors and reconnects	User action	Live video stream
2	Frame	Sample frames, downscale	Video frame	Small grayscale

2	Frame Preprocessor	Sample frames, downscale, convert to grayscale, blur	video frame	Small grayscale pixel buffer
3	Motion Detection Engine	Compare frames and compute a motion score	Current + previous buffer, config	Motion score, changed-pixel ratio
4	Motion State Machine	Convert raw scores into stable states (debounce and cooldown)	Motion score	State change events
5	Render Controller	Apply black-and-white or color to the display	State events	Styled video output
6	Alert Manager	Show banner, play sound, send browser notification	MOTION_STARTED event	Alerts
7	Event Logger	Record events, optionally snapshot	State events	Log entries, snapshots
8	Settings Manager	Hold sensitivity, cooldown, alert options	UI input	Config object
9	UI Layer	Controls, status, history	All events	User actions
10	Sync Service (optional)	Send events and snapshots to backend	Log entries	API calls

5.2 Component Details

5.2.1 Camera Capture Module

- Asks the browser for webcam access at a moderate resolution (about 640 × 480 at 30 FPS) with no audio.
- Handles these situations: permission denied, no camera found, camera already in use, and stream ended unexpectedly.
- Offers start, stop and switch-camera actions, and announces "camera ready" or "camera error" to the rest of the system.
- Pauses the stream when the tab is hidden, to save resources.

5.2.2 Frame Preprocessor

Reduces data to make detection fast and stable.

1. Copy the current video frame onto a hidden **small image area** (for example 160 × 120 pixels).
2. Read the pixel values.
3. Convert color to shades of gray using a weighted average of red, green and blue (green counts most, as the eye is most sensitive to it).
4. Optionally, apply a light blur to remove camera sensor noise.

4. Optionally apply a light blur to remove camera sensor noise.

Detection runs on this small hidden copy. The video the user sees is full resolution and is **not** modified by this step.

5.2.3 Motion Detection Engine

Implements **frame differencing with an adaptive background**:

1. For each pixel, measure how much it differs from the reference image of the still scene.
2. A pixel counts as "changed" if the difference is larger than the pixel threshold.
3. The **motion score** is the share of pixels that changed.
4. Motion is flagged if the motion score is larger than the area threshold.
5. The reference image is updated slowly over time, so gradual lighting changes are absorbed.

The detailed procedure is in Section 8.

5.2.4 Motion State Machine

Prevents flicker between black-and-white and color (Section 7).

5.2.5 Render Controller

Two approaches, chosen for simplicity and efficiency:

Approach	How	Pros	Cons
CSS filter (recommended)	Apply or remove a grayscale visual effect on the video element, with a short fade	GPU accelerated, almost zero CPU, smooth fade	Display only; recording would need canvas
Canvas rendering	Draw frames to a visible canvas, converting to gray when idle	Required if recording or capturing the processed stream	More CPU use

Default: CSS filter. Use canvas only if the client needs the black-and-white effect baked into recordings or snapshots.

5.2.6 Alert Manager

Supports multiple channels, each switchable in settings:

- **On-screen banner**: "⚠ Motion detected at 14:32:10" with a red border on the video.
- **Sound**: short beep through the Web Audio API (needs one prior user interaction because of autoplay rules).
- **Browser Notification**: via the Notifications API, useful when the tab is in the background.
- **Remote notification (optional)**: email, SMS or push via backend.
- Rate-limited so the user is not flooded (for example, at most one alert per N seconds).

5.2.7 Event Logger

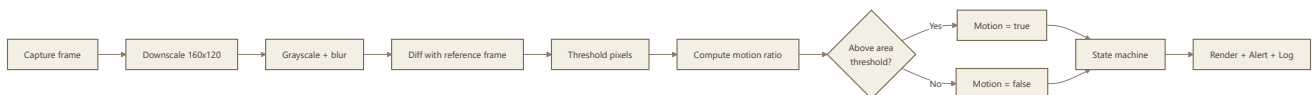
Stores: eventId, startTime, endTime, duration, peakScore, optional snapshot (data URL / blob). Stored in **IndexedDB** with a capped size (for example 500 events, oldest removed first).

5.2.8 Settings Manager

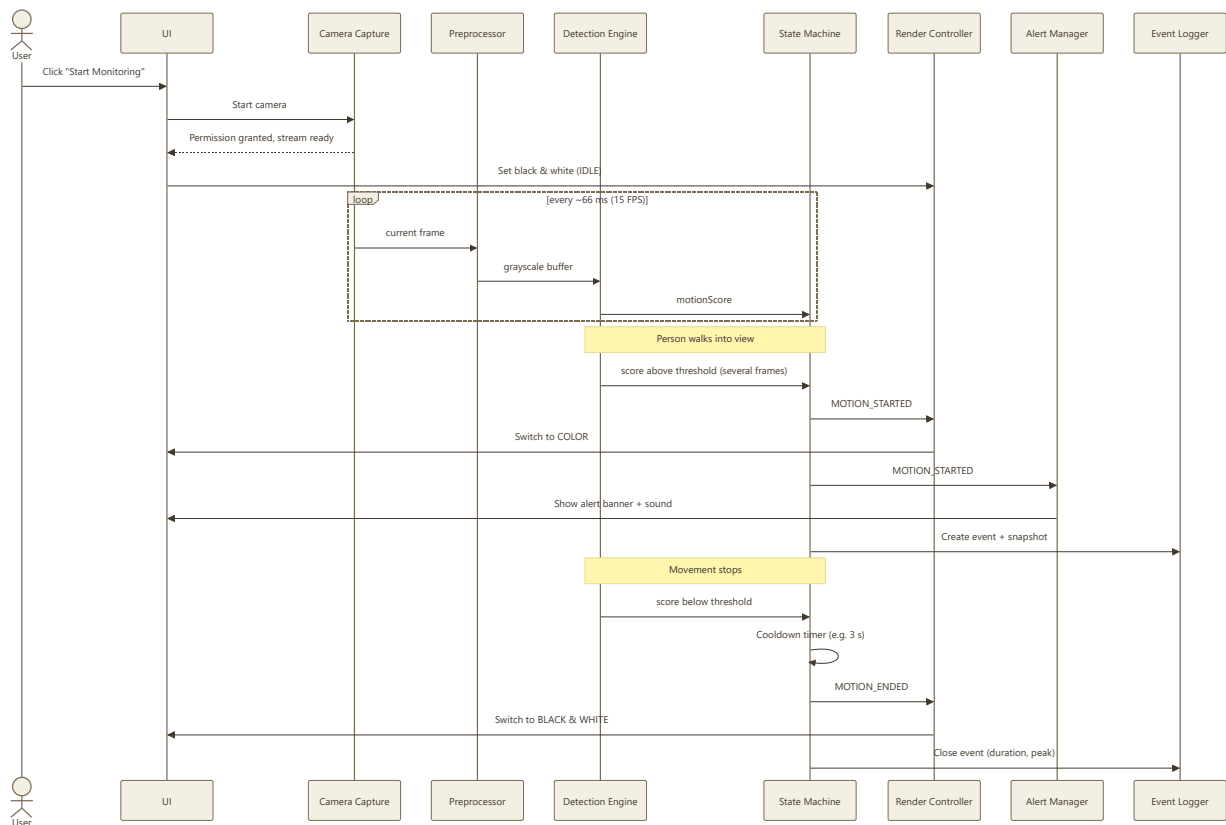
Saves configuration on the device (browser local storage), validates the allowed ranges, and notifies the other modules when a setting changes.

6. Runtime Behavior

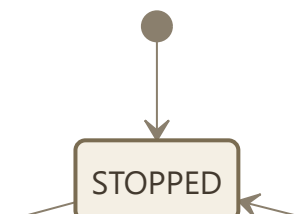
6.1 Processing Pipeline

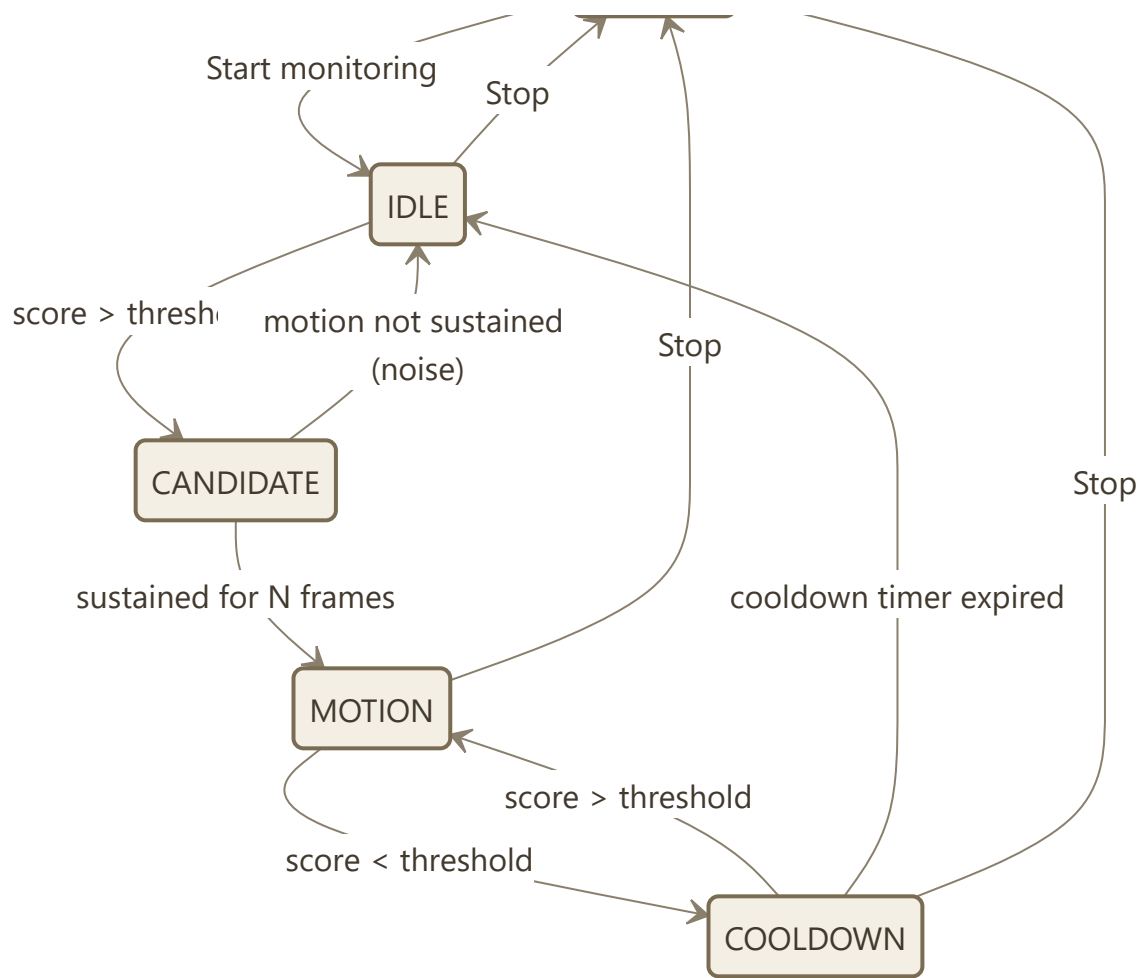


6.2 Sequence Diagram: Motion Occurs



7. Motion State Machine





State	Display	Alert	Description
STOPPED	Placeholder	None	Camera is off
IDLE	Black & white	None	Watching, nothing moving
CANDIDATE	Black & white	None	Possible motion; confirming over a few frames (debounce)
MOTION	Color	Active	Motion confirmed
COOLDOWN	Color	Cleared after motion event ends	Waiting a few seconds before returning to black and white

This design guarantees that a single noisy frame never triggers an alert and the display does not flicker.

8. Motion Detection Algorithm

8.1 Detection Procedure (step by step)

Tunable parameters

Parameter	Default	Meaning
Processing size	160 × 120 pixels	Size of the small image used for comparison
Pixel threshold	25 (on a 0–255 scale)	How much a single pixel must change to count as "changed"
Area threshold	2%	Share of the image that must change to count as motion
Confirm frames	3	Consecutive frames needed before motion is confirmed (debounce)
Cooldown	3 seconds	Delay after motion stops before returning to black and white
Background learning rate	5%	How quickly the reference image adapts to the scene

For every processed frame

1. Shrink the frame to the processing size and convert it to grayscale; apply a light blur.
2. If there is no reference image yet, save this frame as the reference and wait for the next one.
3. Compare every pixel with the reference image and count how many changed by more than the pixel threshold.
4. Calculate the motion score: changed pixels divided by total pixels.
5. Blend the current frame slightly into the reference image, so slow lighting changes are absorbed.
6. If the motion score is above the area threshold, report "motion" for this frame; otherwise report "no motion".
7. Pass the result to the motion state machine (Section 7), which decides when to confirm motion, raise the alert and switch the display.

8.2 Robustness Measures

Problem	Mitigation
Camera sensor noise	Gaussian blur + pixel threshold
Gradual lighting change (clouds, sunset)	Adaptive background (running average)
Sudden light flicker or auto-exposure	Ignore frames where average brightness shifts abnormally
Tiny movements (dust, insects)	Minimum area threshold
Single-frame glitches	Confirm over N consecutive frames
Alert spam	Cooldown and alert rate limiter

Region of no interest (TV, window)

Optional **region of interest (ROI)** mask

8.3 Performance Design

- Process at **10–15 FPS**, not at the full 30–60 FPS; motion detection does not need more.
 - Synchronize processing with the video's own frame timing where the browser supports it; otherwise use a timer-based gate.
 - Run the heavy image work in a **background worker** (off the main page thread) so the interface stays smooth.
 - Reuse the same memory buffers for every frame to avoid slowdowns caused by memory cleanup.
 - Optional upgrade: WebAssembly or WebGL shader for diffing on high-resolution streams.
-

9. Data Design

9.1 Configuration Settings

Setting	Default	Notes
Sensitivity	5 (scale 1–10)	One slider that controls both thresholds below
Pixel threshold	25	Set automatically from sensitivity
Area threshold	2%	Set automatically from sensitivity
Confirm frames	3	Debounce
Cooldown	3 seconds	User adjustable
Processing speed	15 frames per second	Lowered automatically on slow devices
Alerts	Banner on, sound on, notification off	Each channel switchable
Snapshot on motion	On	Optional
Region of interest	Whole image	Optional area to watch

A single **sensitivity slider (1–10)** controls the pixel and area thresholds, so non-technical users can tune it easily.

9.2 Motion Event Record

Field	Example	Description
Event ID	evt_20261004_143210	Unique identifier

Start time	2026-10-04 14:32:10	When motion was confirmed
End time	2026-10-04 14:32:19	When the display returned to black and white
Duration	9 seconds	End minus start
Peak score	31%	Highest share of the image that changed
Snapshot	Photo (optional)	Image taken when motion started

9.3 Storage Strategy

Data	Location	Reason
Settings	localStorage	Small, simple
Event log and snapshots	IndexedDB	Handles larger blobs
Video frames	Not stored (memory only)	Privacy and efficiency

10. User Interface Design

10.1 Screen Layout

Area	Content
Header	Application name, status badge (Monitoring / Idle / Motion), settings button
Main panel	Large live video (black and white or color); alert banner overlaid with the message "Motion detected" and the time
Control bar	Start, Pause and Stop buttons; sensitivity slider
Recent events	List of past motion events with time, duration and snapshot thumbnail

10.2 UX Details

- **Status badge:** Idle (grey), Motion (red, pulsing).
 - Smooth 200 ms transition between grayscale and color.
 - Clear permission-denied screen with instructions to enable the camera.
 - Responsive layout for desktop, tablet and phone.
 - Accessible: alert text is announced by screen readers, and the state is never indicated by color alone.
-

11. Technology Stack

11.1 Phase 1: Client Only

Concern	Technology
Language	JavaScript or TypeScript (TypeScript recommended)
UI	React (or plain HTML/CSS/JS for a lightweight build)
Build tool	Vite
Camera	WebRTC (browser camera access)
Image processing	Canvas 2D / OffscreenCanvas, Web Worker
Alerts	Web Audio API, Notifications API
Storage	localStorage, IndexedDB (via idb)
Hosting	Any static HTTPS host (Netlify, Vercel, S3 + CloudFront, Nginx)
Testing	Vitest/Jest (logic), Playwright (end-to-end)

11.2 Phase 2: Optional Backend

Concern	Technology
API	Node.js (Express / Fastify) or Python (FastAPI)
Realtime	WebSocket or Server-Sent Events
Database	PostgreSQL (events), S3-compatible storage (snapshots)
Auth	JWT / OAuth 2.0
Notifications	Email (SES/SendGrid), SMS (Twilio), Web Push
Hosting	Docker containers on a cloud provider

11.3 Alternative: Python Desktop Version

If the client needs a **desktop app** instead of a web app: Python + the **OpenCV** image-processing library with a desktop user interface (Tkinter or PyQt). The same architecture (capture, preprocess, detect, state machine, render, alert) applies unchanged.

12. Module Organization

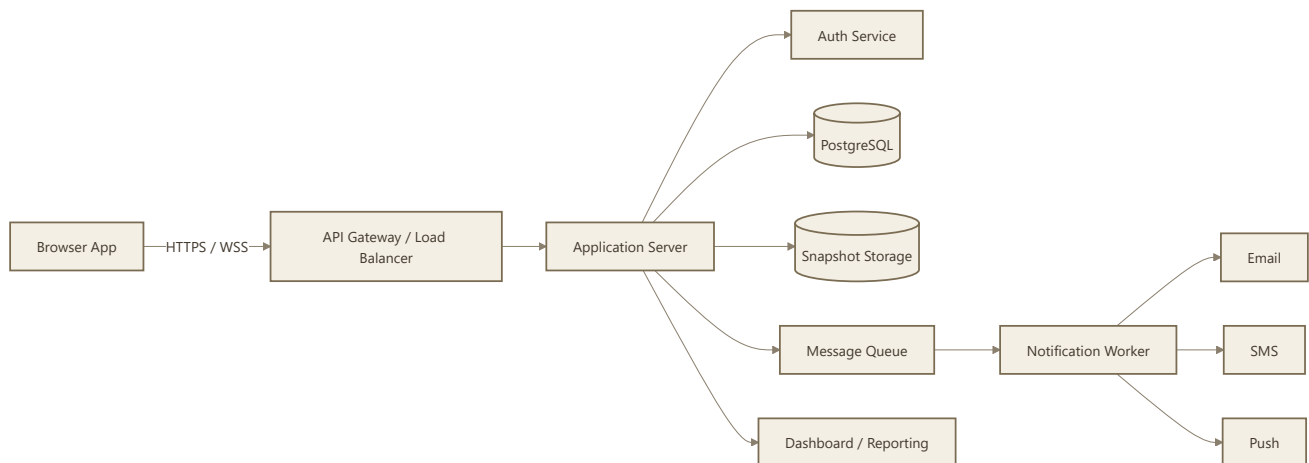
The application code is organized into clearly separated groups, matching the components in Section 5.

Group	Contains	Purpose
Core	Event bus, motion state machine,	Coordination and shared rules

	configuration	
Camera	Camera service	Webcam access and error handling
Processing	Frame preprocessor, motion detector, background worker	Image analysis
Output	Render controller, alert manager, event logger	What the user sees and hears; record keeping
User interface	Main screen, video panel, controls, settings panel, event list	Presentation
Storage	Local database access	Saving settings, events and snapshots
Assets	Alert sound and static files	Resources
Tests	Unit, integration and end-to-end tests	Quality assurance

Pure logic (processing and state machine) is kept separate from the user interface so it can be tested on its own and reused.

13. Optional Backend Architecture (Phase 2)



Backend capabilities (example)

Capability	Purpose
Upload motion event	Store an event's time, duration and score
Upload snapshot	Attach a photo to an event
List events	Retrieve events for a date range
Read / update settings	Sync settings across devices

Sign in

Authenticate the user

Only **metadata and snapshots** are sent, never a continuous video stream, which keeps bandwidth and privacy impact low.

14. Cross-Cutting Concerns

14.1 Security and Privacy

- Serve exclusively over **HTTPS** with HSTS; set a strict Content-Security-Policy.
- Process video locally by default; show a clear camera-active indicator.
- Any upload is **opt-in**, with explicit consent text.
- Backend: TLS, authenticated APIs, encrypted storage, signed URLs for snapshots, retention policy (auto-delete after N days).
- Comply with applicable regulations (GDPR/CCPA) if recording people; add a privacy policy.

14.2 Error Handling

Error	Handling
Permission denied	Friendly message plus steps to re-enable
No camera	Show "No camera detected" and retry button
Camera disconnected	Detect that the camera stream ended, attempt reconnect, notify user
Low FPS	Auto-reduce processing resolution
Storage full	Remove the oldest events
Notification blocked	Fall back to banner and sound

14.3 Observability

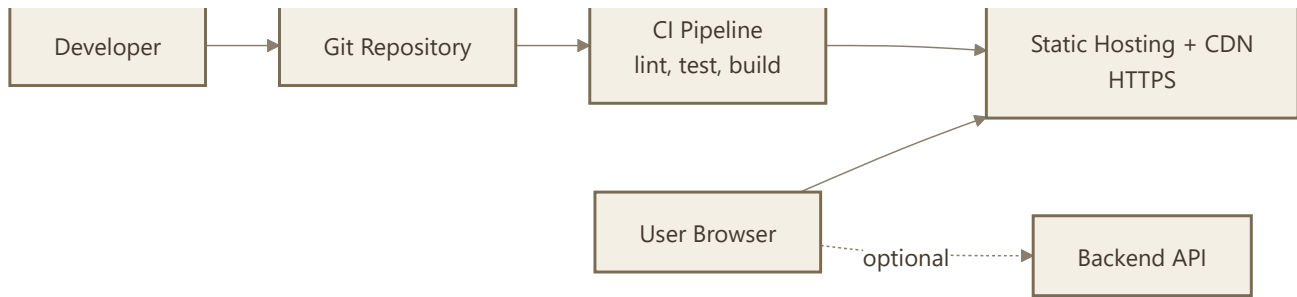
- Client: lightweight error reporting (for example Sentry), optional anonymous performance metrics (FPS, detection time).
- Backend: structured logs, health check endpoint, metrics and alerts.

14.4 Accessibility and Internationalization

- Keyboard-operable controls, ARIA live regions for alerts, sufficient contrast.
 - Text externalized for translation.
-

15. Deployment Architecture





- **Phase 1** needs only static hosting with a CDN and a TLS certificate. This keeps hosting cost very low.
- CI/CD: build and test on every commit; deploy automatically to staging, then production after approval.
- Environments: Development, Staging, Production.

16. Testing Strategy

Level	What is tested	Tools
Unit	Grayscale conversion, differencing, state machine transitions, sensitivity mapping	Vitest/Jest
Integration	Pipeline with recorded video fixtures (still, slight noise, walking person, lighting change)	Vitest + sample frames
End-to-end	Permission flow, start/stop, B&W ↔ color switch, alert display (using the browser's built-in fake camera feature)	Playwright
Performance	FPS and CPU on low-end devices	Browser profiler
Cross-browser	Chrome, Edge, Firefox, Safari, mobile browsers	BrowserStack or manual
Accuracy	False positive / false negative rate over test videos	Custom script

Acceptance criteria

- Still scene for 60 s: **0 alerts**, display stays black and white.
- Person enters view: color and alert within **300 ms**.
- Movement stops: black and white returns after the configured cooldown.
- No visible UI lag at 15 FPS processing.

17. Risks and Mitigations

Risk	Impact	Mitigation
False alerts from	Approved users	Adaptive background brightness, shift filter sensitivity